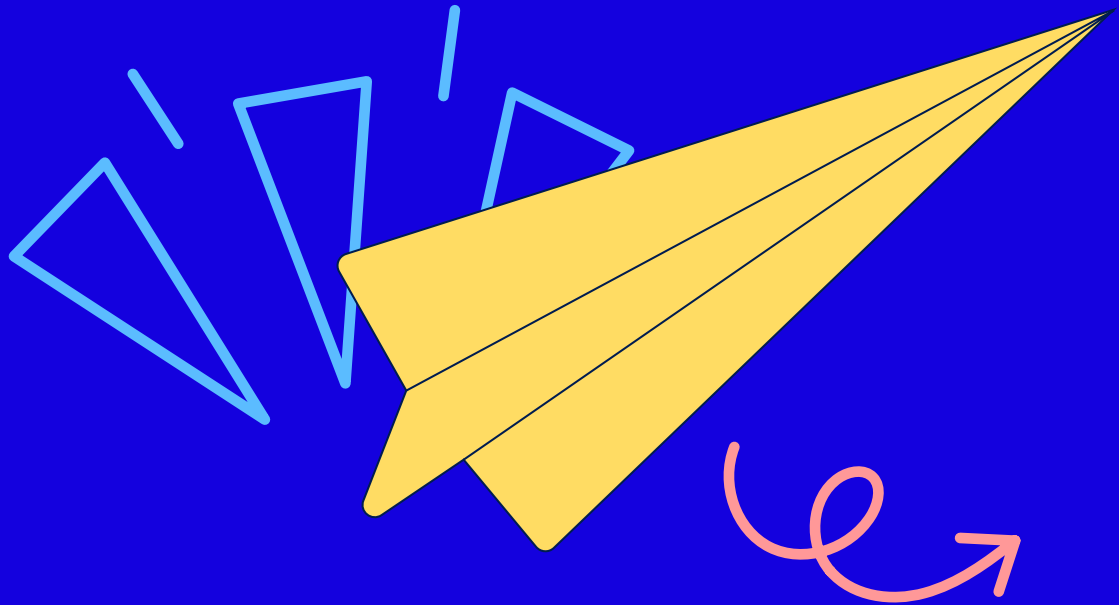




Web components II

Frontend



Web components II

Hasta ahora hemos visto una introducción a los *web components*, concretamente, hemos aprendido y practicado con cada una de las tecnologías que componen los *web components*: HTML Templates, Custom Elements y Shadow DOM. Ya sabemos también cómo se conjugan para crear estos componentes web que podemos reutilizar múltiples veces y en múltiples sitios web sin causar ningún tipo de afectación a su entorno y tampoco sufrirla, lo que nos permite un sinfín de posibilidades. También hemos visto cómo podemos personalizar estos componentes web a través de atributos y a través de variables CSS.

Ahora toca profundizar algo más en su funcionamiento y, sobre todo, entender cómo podemos **comunicar los web components entre sí**. La comunicación es fundamental, de ello depende la modificación de propiedades, los cambios de estado, la reactividad y una larga lista de funcionalidades indispensables para la construcción de aplicaciones web.

Autor: Miguel Ángel Muñoz



Comunicación



Lit



¿Qué hemos aprendido?

Comunicación

Como ya sabes, disponemos de los atributos de los *web components* a través de los cuales el propio elemento puede reaccionar al cambio si así lo configuramos.

Atributos

De aquí podemos extraer un tipo de comunicación que se denomina *Parent-to-Child*. El componente padre puede realizar un cambio en un atributo del componente hijo para que el componente hijo reaccione de la forma que se configure. El componente hijo, al estar escuchando los cambios que se producen en ese atributo, puede realizar acciones al respecto.

Si tomamos como ejemplo un formulario en el que el componente padre es el *form* y el componente hijo es el *input* (aunque serían *web components* personalizados), podría darse el caso en el que el componente padre (*form*) quisiera indicar un cambio de valor al componente hijo (*input*), el cual podría realizarlo a través de la modificación de un atributo expuesto. Por ejemplo, si al interactuar con un componente de tipo *radio button* queremos que se autorellene un componente de tipo *input* automáticamente.

Mediante eventos

Ya conocemos los eventos y sabemos que JavaScript se basa en eventos, contando así con las siguientes posibilidades:

- ☐ Emitir eventos.
- ☐ Escuchar eventos.
- ☐ Usar eventos predefinidos por defecto.
- ☐ Crear eventos customizados.

Esto nos aporta una cantidad inmensa de posibilidades, sobre todo porque podemos crear nuestros propios eventos y configurarlos en función de nuestras necesidades.

1

Eventos personalizados

Para crear un evento personalizado únicamente tenemos que crear una nueva instancia del objeto *Event* de la siguiente manera:

```
const myCustomEvent = new Event('bubblingTest')
```

Lo único que tenemos que indicar como parámetro obligatorio es el nombre que le queremos dar al evento, en este caso *'bubblingTest'*.

Además, disponemos de varias opciones que nos permiten configurar nuestro evento para adaptarlo a nuestras necesidades:

- **bubbles:** un booleano (*true o false*) que indica si queremos que se propague a lo largo del árbol del DOM. Lo veremos con más detalle un poco más adelante.
- **cancelable:** un booleano que indica si queremos permitir que se pueda cancelar la emisión del evento utilizando *event.preventDefault()*.
- **composed:** un booleano que indica si queremos que el evento atraviese el *Shadow DOM*.
- **detail:** permite añadir información extra al evento pudiendo estructurarse como un objeto.

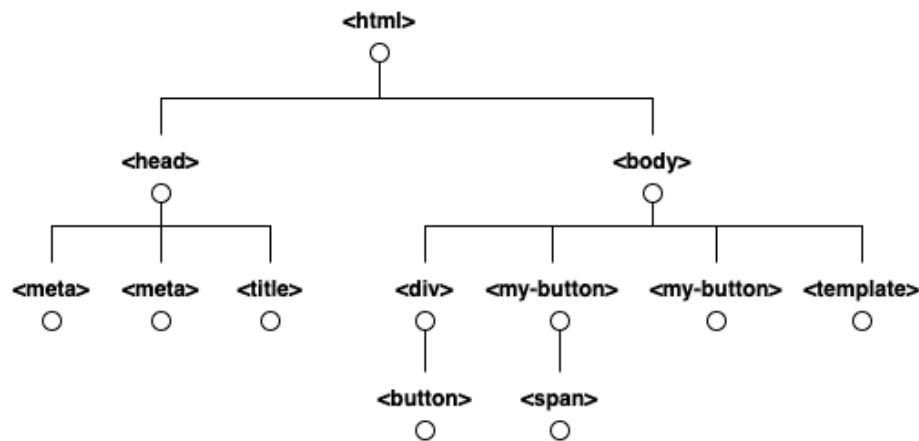
Teniendo en cuenta estas opciones, podemos hacer una configuración de nuestro evento de la siguiente manera:

```
const myCustomEvent = new Event('bubblingTest', {  
  bubbles: true,  
  composed: true  
});
```

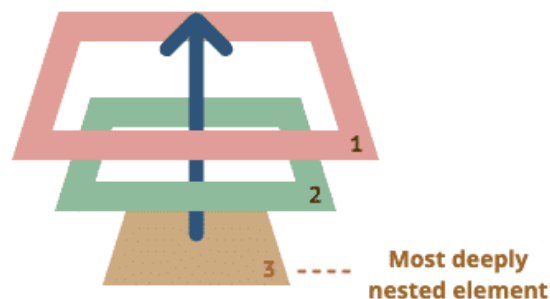
Por defecto, los valores de todas las opciones de nuestros eventos personalizados se configuran a *false*. En este caso estamos habilitando el *bubbling* y el *composed*.

Esta característica es uno de los puntos fuertes de JavaScript, es lo que le da toda su potencia. Pero... ¿Qué es el *Event Bubbling*?

Como ya hemos visto en otro momento, el DOM se representa como un árbol en el que existen padres e hijos:



El *Event Bubbling* viene a permitir que los eventos que se producen suban al igual que hacen las burbujas en un vaso de agua, por ejemplo. Cuando sale una burbuja en la base o en algún punto intermedio del vaso con agua gaseosa, sube hacia arriba hasta la superficie. El Event Bubbling se comporta de la misma manera: si un evento que se produce en un elemento (un *clic* o un *custom event*, por ejemplo) tiene configurada a *true* la propiedad de *bubbles*, subirá hacia arriba en el árbol, permitiendo que los elementos padre puedan escuchar el evento por medio del método *addEventListener* del elemento.



Podemos obtener el origen del evento, es decir, el elemento que emitió ese evento en origen dentro del DOM. Para ello podemos consultar la propiedad `target` del evento con `event.target`.

También podemos parar la propagación de un evento, por ejemplo: si queremos realizar una comunicación entre hijo y padre pero no deseamos que suba más allá, el elemento padre podría parar la propagación utilizando el método `stopPropagation` del evento:

```
event.stopPropagation();
```

3

Emitiendo y escuchando eventos

Ahora que ya sabemos qué son los eventos personalizados y el *bubbling*, podemos usarlos conjuntamente para establecer una comunicación entre elementos.

Podemos activar la propiedad *bubbles* para que se propague hacia arriba hasta el final o hasta que paremos la propagación:

```
const bubblingEvent = new Event('bubblingTest', {
  bubbles: true,
  composed: true
});
```

Después, definimos el evento personalizado con las propiedades *bubbles* y *composed* a *true*.

```
clickBubblingTestChild.addEventListener('click', () => {
  bubblingTestChild.dispatchEvent(bubblingEvent);
});
```

Aquí tengo puesto un botón que cuando se haga clic emita un evento en el elemento *bubblingTestChild* con el objetivo de que el padre capture su evento.

Por otro lado, podemos definir un evento con las propiedades *bubbles* y *composed* a *false* y emitir el evento directamente en el padre:

```
const bubblingEvent = new Event('bubblingTest', {  
  bubbles: false,  
  composed: false  
});
```

Y ahora hacemos el *dispatchEvent* sobre el padre en lugar de sobre el elemento hijo:

```
clickBubblingTestChild.addEventListener('click', () => {  
  bubblingTestChild.parentElement.dispatchEvent(bubblingEvent);  
});
```

Esto conlleva un problema, que en algún momento estés en el Shadow DOM y el elemento padre esté en el DOM, y por tanto tendrás que utilizar esto:

```
hiddenDiv.getRootNode().host.dispatchEvent(bubblingEvent);
```

***hiddenDiv* es el elemento que se encuentra en el Shadow DOM y su padre es el Shadow Host, por lo que es necesario utilizar el método *getRootNode()* que nos devuelve el Shadow Root y obtener la propiedad *host* para conseguir el Shadow Host.**

Contexto

Ya habrás estudiado en JavaScript sobre los *scopes*, es decir, sobre los diferentes contextos que hay a nivel de bloque. También sabrás que existe un contexto global que es el *window*.

Podemos utilizar este contexto global (siempre con mucho cuidado) para guardar información que necesitamos que sea expuesta a nivel global del sitio web. Es muy común que los *frameworks* como **Angular** o **React** utilicen este contexto global para guardar propiedades que necesitan que estén expuestas para ser usadas por sus elementos.

 En **React**, por ejemplo, existe el [React Context API](#).

Podéis probar a crear un nuevo objeto dentro del contexto global con el nombre de vuestro *framework* personalizado y ahí declarar funciones para gestionar este contexto. Por ejemplo:

```
window.globalContext = {  
  name: 'globalContext',  
  sayHello: function() {  
    console.log(`Hello from ${this.name}`);  
  }  
};
```

A partir de que se declare este objeto ya está disponible este contexto de manera global, por lo que se puede consultar de la siguiente manera:

```
window.globalContext.name
```

O directamente con:

```
globalContext.name;
```

Ya que por defecto va a ir a buscarlo al *scope global*, que es el *window*. Volviendo al ejemplo de *React*, cuando utilizamos *React.createContext('light')*, realmente está buscando en el *window* con *window.React.createContext('light')*.

Esté patrón nos permite compartir información **global**, pero no es algo que debamos utilizar habitualmente, de hecho, siempre tendrá prioridad el resto de los patrones. Además, no es reactivo por sí solo, es decir, si se produce cualquier cambio en el contexto, los componentes no se enteran del cambio a no ser que se utilice paralelamente algún otro patrón, como el observador o el *PubSub* (*publisher - subscriber*), de tal forma que los elementos se suscriben a los cambios que se producen en el contexto y el contexto notifica dichos cambios, aunque esto se aproxima más a lo que denominamos '*un gestor de estados*'.

Gestor de estados

Podríamos definir estado como el conjunto de todos los valores almacenados por la aplicación por medio de propiedades o variables en cualquier momento de ejecución.

Este estado puede contener respuestas que obtengamos de las APIs, propiedades globales o locales seteadas por algún elemento de la aplicación, estado de navegación y un largo etcétera.

Lo que buscamos con el estado es representar la situación en la que se encuentra la aplicación en cada momento, pudiendo llegar al extremo de poder recuperar dicho estado tras un reinicio de la aplicación.

La característica esencial de un gestor de estados es, como su propio nombre indica, dotar de la capacidad de gestionar dicho estado, es decir, poder modificarlo, consultarlo y reaccionar frente a los cambios que se produzcan.

Para conseguir hacer un gestor de estados podría bastarnos, como mencionamos anteriormente, con usar el contexto y algún patrón que nos permita notificar y ser notificados cuando se produce cualquier cambio y, por supuesto, realizar modificaciones sobre el mismo.

Ahora bien, si queremos mantener el estado global de una aplicación más compleja, probablemente lo más recomendable sea utilizar alguna herramienta especializada en gestión de estados como [Redux](#) que nos permite registrar todo cambio que se produzca, tanto síncrono como asíncrono, guardando cuándo, cómo y por qué del cambio del estado.

Patrón observador

El patrón observador se compone de un sujeto (*Subject*) y un observador (*Observer*) o múltiples observadores. El *Subject* es el que almacena el estado y lanza un evento cada vez que se produce cualquier actualización para que los *Observers* reciban dicha actualización y puedan actualizar en consecuencia. Obviamente, este patrón se puede complicar todo lo que queramos aportando utilidades en función de nuestras necesidades, pero la base es esta, lo que nos permite estar actualizados de cualquier cambio de un estado siempre que nos mantengamos suscritos.



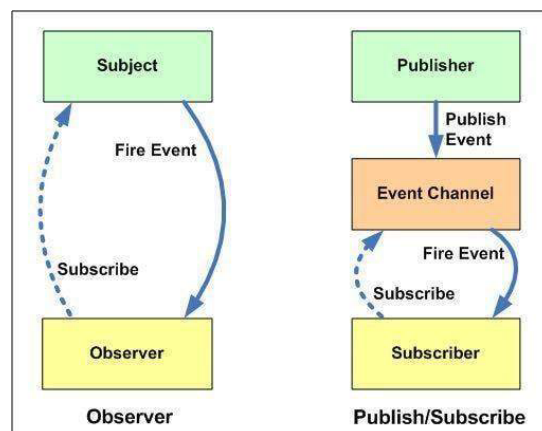
Angular, por ejemplo, se basa en este patrón para mantener las aplicaciones actualizadas; de hecho, ahora ha sacado un *wrapper* o encapsulado para el uso de este patrón de una forma más simplificada, y se denomina **Angular Signals**.



La librería por excelencia para la implementación en frontal de este patrón es **RxJS** que es muy completa y permite realizar verdaderas virguerías.



Un patrón muy similar es el **PubSub** o **Publisher/Subscriber**, que, en este caso, la principal diferencia es la existencia de un canal de eventos, un intermediario que se encarga de gestionar las actualizaciones, registra los suscriptores y les emite la actualización cuando el publicador le comunica un cambio. Aquí no se conocen el *Publisher* y los *Subscribers*, en cambio, en el patrón observador, el *Subject* lleva un registro de todos los observadores a los que tiene que notificar el cambio.



En resumen, mediante estos patrones podemos tener un contexto del que además podemos estar continuamente actualizados.

Conclusiones

Hemos visto diferentes patrones y métodos que podemos utilizar para realizar comunicaciones entre los diferentes componentes. Como todo en programación, su uso depende de cada situación, no hay ninguno mejor que otro a nivel general, hay unos mejores que otros en cada situación.

Por regla general para hacer comunicaciones entre padres e hijos podemos utilizar los atributos de los componentes y la comunicación mediante eventos, y para comunicación de estados globales, podemos utilizar el contexto global, los gestores de estados o los patrones observadores y *Publisher/Subscriber*.

Por regla general, para hacer *comunicaciones entre padres e hijos* podemos utilizar los atributos de los componentes y la comunicación mediante eventos, y para *comunicación de estados globales*, podemos utilizar el contexto global, los gestores de estados o los patrones observadores y *Publisher/Subscriber*.

Lit

Lit es una librería como ya he mencionado y no hay que confundirlo con un *framework* como Angular, React o Polymer. La diferencia fundamental es que, tras utilizar la librería, vamos a recibir como resultado un componente nativo como los que estábamos construyendo hasta ahora, sin atarnos a ningún *framework*, lo que significa que es completamente funcional en cualquier entorno por el hecho de ser nativo.

**La librería de Lit nos facilita ciertas tareas como la gestión de los atributos, el rendering, el lifecycle...
Descubramos sus bondades.**

Instalación

El cambio principal es que en lugar de extender la clase de nuestro *web component* de *HTMLElement*, que es una clase nativa, lo vamos a extender de la clase *LitElement* y, como no es una clase nativa, necesitamos añadirla a nuestras dependencias. Para ello, al igual que hemos instalado otras dependencias utilizaremos el siguiente comando:

```
npm i lit
```

Una vez instalado, únicamente tenemos que importar la clase:

```
import { LitElement } from 'lit';
```

A partir de ahí ya podríamos empezar a crear nuestro componente.

Estructura de un componente

Para crear un componente con Lit, la estructura es muy similar a como hemos estado creando los componentes de manera nativa, aunque con algunas ventajas que lo simplifican.

La definición de la clase se hace de forma muy similar, con la excepción que comentábamos antes de que tenemos que extender de la clase *LitElement*:

```
import { LitElement, html, css } from 'lit';

export class CardComponent extends LitElement { ... }

window.customElements.define('card-component', CardComponent);
```

Si te fijas en la documentación de Lit te propondrá el uso de **decoradores** como `@customElement`, pero **están enfocados a TypeScript, no a JavaScript, salvo que incluyamos un compilador como Babel.**

A la hora de definir tanto los estilos como el HTML, Lit nos proporciona dos utilidades que nos simplifican su definición.

- **Estilos**

css es una utilidad que nos permite definir los estilos directamente sin tener que englobarlos dentro de una etiqueta *style*. Además, nos ofrece la posibilidad de concatenar múltiples bloques de estilos.

```
css`
  .card-component__container {
    padding: var(--card-component-padding);
    border-radius: var(--card-component-border-radius);
    background-color: var(--card-component-color);
    height: var(--card-component-height);
  }
`;
```

La definición es sencilla, añadimos la palabra `css` seguida de comillas invertidas (las mismas que utilizamos para definir las *Template Literals*).

Para definir los estilos del componente los tenemos que asignar a la propiedad [static styles](#). Esta propiedad, al igual que la de *properties* que veremos más adelante, son *static* para que puedan ser llamadas sin necesidad de instanciar una clase, así se pueden obtener los estilos y las propiedades directamente.

```
static styles = css`
  .card-component__container {
    padding: var(--card-component-padding);
    border-radius: var(--card-component-border-radius);
    background-color: var(--card-component-color);
    height: var(--card-component-height);
  }
`;
```


El valor de la propiedad `styles` puede ser único o múltiple definiendo un *array*:

```
static styles = [
  css`
    .card-component__container {
      padding: var(--card-component-padding);
      border-radius: var(--card-component-border-radius);
      background-color: var(--card-component-color);
      height: var(--card-component-height);
    }
  `,
  css`.card-component__container { padding: 1rem; }`
];
```

¡Ojo! La utilidad CSS no nos permite inyectar variables como hacíamos antes:

```
<style>
  .card-component__container {
    padding: ${this.cardPadding};
    border-radius: ${this.cardBorderRadius};
    background-color: var(--card-component-color);
    height: ${this.cardHeight};
  }
</style>
```

No podemos utilizar propiedades del componente que sean dinámicas, ya que los estilos del componente se procesan una única vez y a nivel global, por lo que son globales para todos los componentes que se creen del mismo tipo.

Podríamos utilizar una constante que definamos en la clase, aunque tendría que ser de tipo `css` para que no nos dé un error cuando vayamos a cargar el *web component*. Es un mecanismo de seguridad para evitar inyección de código. La utilidad `css` se encarga de sanitizar el valor que se le asigne. Si a la constante `cardPadding` le asignamos un valor de tipo `css` nos lo dará por bueno, fíjate bien:

```
const cardPadding = css`red`;
```

Otra opción que tenemos, pero que no deberías usar, es la utilidad `unsafeCSS` que permite marcar una variable como *unsafe* e indicar que aceptamos que así sea.

```
const cardPadding = `1rem`;

static styles = css`
  .card-component__container {
    padding: ${unsafeCSS(cardPadding)};
    border-radius: var(--card-component-border-radius);
    background-color: var(--card-component-color);
    height: var(--card-component-height);
  }
`;
```

 Te recomiendo que profundices en la [documentación](#), donde encontrarás más utilidades interesantes a la hora de crear tus estilos: estilado del elemento padre, creación de mapas de clases y de estilos, etc.

- **Propiedades**

Las propiedades en Lit se pueden dividir en dos tipos:



Propiedades públicas reactivas: son expuestas y modificables desde fuera a través de los atributos (como ya hemos visto de manera nativa), y además se emiten eventos cuando son modificadas.



Propiedades internas: no se exponen y son de carácter privado, por lo que no se les asocia un atributo para que puedan ser modificadas, no son accesibles desde fuera.

De nuevo, para **definirlas**, hay dos formas: mediante el uso de decoradores o mediante la definición de la propiedad *static properties*.

```
static properties = {  
  cardBorderRadius: { type: String },  
  cardHeight: { type: String },  
  cardPadding: { type: String }  
};
```

Para inicializar estas propiedades debemos hacerlo en el *constructor* directamente.

```
constructor() {  
  super();  
  this.cardBorderRadius = '0.5rem';  
  this.cardHeight = '27rem';  
  this.cardPadding = '1rem';  
}
```

No hay que definirlas como propiedades de clase, ya que si no nos dará problemas en la reactividad.

Las propiedades disponen de múltiples **opciones de configuración**:



converter: permite definir cómo queremos transformar el tipo de dato que recibimos del atributo (*fromAttribute*) y como queremos transformar el tipo de dato que enviamos al atributo (*toAttribute*).



reflect: es un booleano (por defecto *false*) que permite indicar si queremos que el valor de la propiedad se refleje en el atributo cada vez que esta cambie.



type: nos permite definir el tipo que queremos que tenga la propiedad (por defecto *String*).



attribute: booleano (por defecto a *true*) que indica si queremos asociar la propiedad con un atributo o no para que sea expuesta. Si se define como *false*, se ignoran las propiedades: *converter*, *reflect* y *type*. El valor de esta opción **también puede ser un *String***, lo que permite *settearla* a *true* y además **definir el nombre del atributo** que se expone.



state: es el booleano que nos permite definir la propiedad como interna. Se recomienda que el nombre de este tipo de propiedades vaya precedido de un guion bajo para indicar que es privada (*_myProperty*).



hasChanged: es la función que se llama cada vez que a la propiedad se le asigna un nuevo valor para determinar si ha sufrido cambios o no. Por defecto se utiliza una regla simple para determinar si hay cambio (*newValue !== oldValue*). Es útil definir esta función de manera personalizada cuando no es tan sencillo determinar si ha habido cambio o no, por ejemplo, podemos tener un listado de valores para los que no queremos que se detecte cambio. Hay que tener en cuenta que, si la función devuelve *true*, sí inicia todo el proceso de actualización del componente; es por este motivo, que en ciertos casos, es fundamental definir esta función.



noAccesor: booleano (por defecto *false*) que permite indicar si queremos que se definan el *setter* y el *getter* de la propiedad para que se pueda interactuar con ella directamente.



Para ver la conversión por defecto puedes ver la tabla que hay en la [documentación](#).

Ciclo de cambios de una propiedad:

Se llama al **setter** de la propiedad para definir un nuevo valor, bien directamente o cambiando el valor del atributo.

El *setter* llama al método **requestUpdate** ya definido por la clase *LitElement*.

Se llama a la función definida para determinar el valor de **hasChanged**.

Si el valor de **hasChanged** es **true**, se pone en cola una nueva actualización del componente a no ser que ya haya alguna en la cola (así se evita que el componente entre en bucle por algún error y sobrecargue el navegador).

Se llama al método **update** definido por la clase *LitElement* para actualizar el componente.



Nuevamente, te recomiendo que profundices en la [documentación](#) sobre las propiedades en Lit.

- **Render**

Ya hemos visto cómo podemos definir los estilos del componente y sus particularidades, también hemos aprendido cómo definir las propiedades y todas las opciones que tenemos para personalizarlas: ahora toca **renderizar el componente** para que podamos ver algo.

En esta parte simplificamos lo que hacíamos de manera nativa, ya que únicamente tenemos que definir un método render y es la clase `LitElement` la que se encarga de llamarlo siempre que corresponda. Además, disponemos de la utilidad `html` que nos permite definir la plantilla HTML de una manera mucho más sencilla:

```
render() {  
  return html`  
    <div class="card-component">  
      <div class="card-component__container">  
        <slot></slot>  
      </div>  
    </div>  
  `;  
}
```

Hay una serie de **normas importantes** que debemos cumplir en la definición de este método:



No hagas cambios de estado, es decir, no cambies el valor de propiedades.

Ten en cuenta que el método render se llama al final de la actualización del componente, es decir, cuando ya se ha determinado que ha habido un cambio y se llama al método **update** que hemos mencionado antes. Si actualizamos nuevamente en el método render entraríamos en un bucle (aunque ya hemos visto antes que quedará protegido).



No provoques ningún efecto secundario. Esto como en cualquier función que defines, pero en este caso con más motivo ya que el método render es sensible.



Utiliza únicamente propiedades del componente como *input* para la toma de decisiones. Si no lo haces, podrías estar generando efectos secundarios, ya que serían propiedades no controladas.



Que el método sea puro, es decir, para los mismos valores de propiedades debería devolver el mismo resultado: insisto, no crees efectos secundarios.

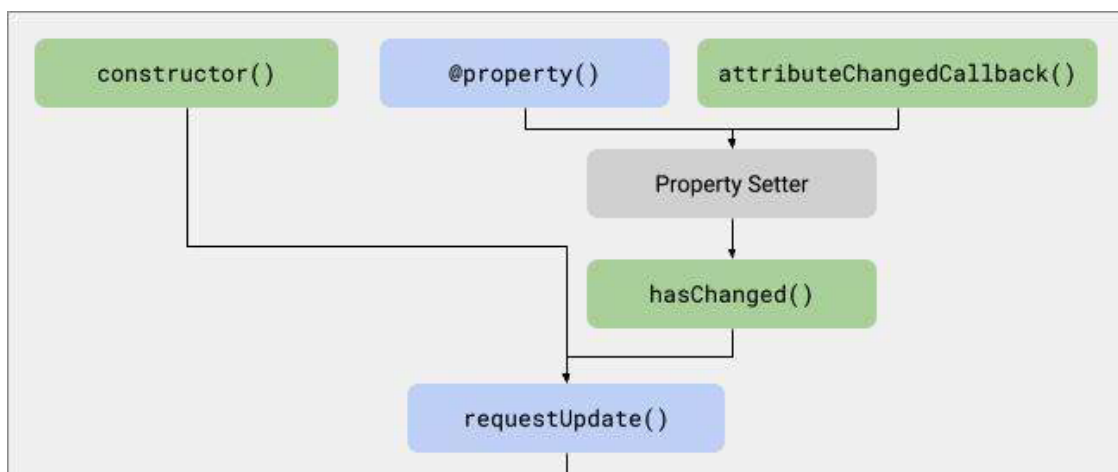


Entra en más profundidad en la [documentación](#).

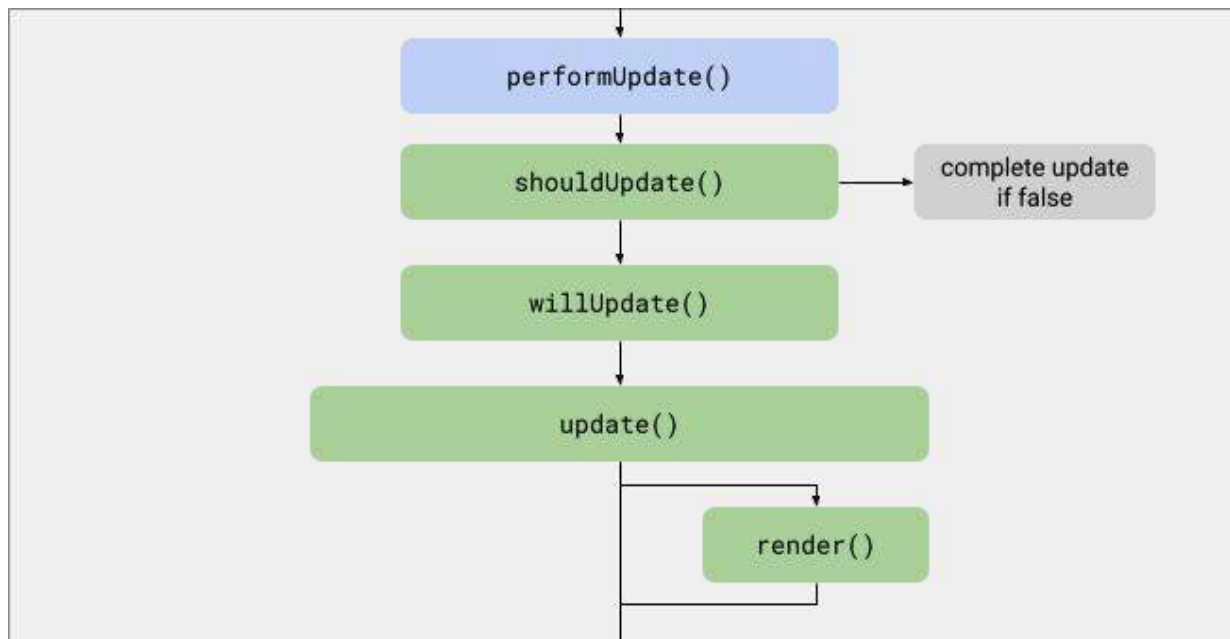
- **Ciclo de vida**

Dentro del ciclo de vida del componente definido por Lit, obviamente disponemos de lo que ya hemos comentado en la parte nativa: constructor, *connectedCallback*, *disconnectedCallback*, *attributeChangedCallback* y *adoptedCallback*.

Como ya hemos ido viendo, sobre todo en la parte de propiedades, hay un ciclo de vida específico de Lit que nos permite, principalmente, **optimizar** la parte de actualización del componente, definir si se debe actualizar o no y evitar que entre en un bucle de renderización.



Esta parte del ciclo de vida ya la hemos comentado anteriormente en la parte de las propiedades, aunque añadimos la parte del constructor, que obviamente se salta el proceso de determinar si se ha producido cambio o no ya que es un cambio *forzado*.



PerformUpdate se encarga de **llamar a diferentes métodos** que se ejecutan antes de realizar la actualización.



ShouldUpdate es el primero de la lista, ya que es el que se encarga de **determinar si es necesaria una actualización** del componente o no. Recibe el mapa de propiedades que han cambiado y sus valores previos. Este método **puede ser personalizado** si lo declaramos en nuestra clase del componente. Puede ser muy útil cuando tenemos casuísticas especiales y no queremos que se produzcan actualizaciones bajo ciertos criterios. Finalmente, **devuelve un booleano** como respuesta para indicar a `PerformUpdate` si se sigue con el ciclo o no.



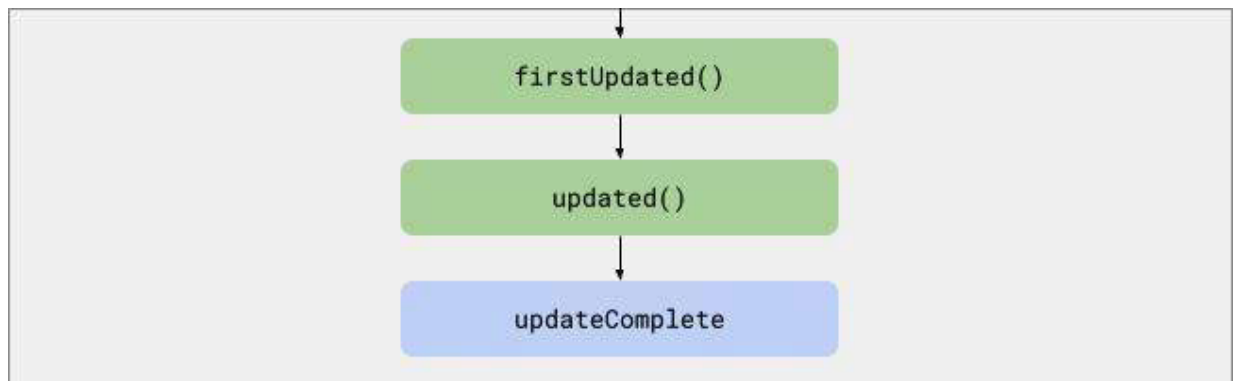
WillUpdate recibe el mismo parámetro que `shouldUpdate` (el mapa de propiedades que han cambiado), y se llama cuando ya se ha determinado que se va a producir una actualización. Nos sirve para **calcular valores** que nos serán útiles de cara a la actualización, por ejemplo, si queremos determinar el valor de una propiedad en base a la conjunción de las propiedades que se hayan actualizado. No devuelve nada, únicamente procesa valores.



Update recibe el mismo parámetro que se va arrastrando durante todo el ciclo de vida y se encarga de **reflejar las propiedades a los atributos**, que así lo tengan configurado, y de llamar al método `render`. Normalmente este método no se personaliza.



Render es el método que ya hemos definido anteriormente.



Ya se ha renderizado el componente, pero antes de cerrar el ciclo de vida, hacemos alguna cosa más.



FirstUpdated recibe el mismo parámetro con el mapa de propiedades actualizadas con sus valores previos, los que venimos arrastrando durante todo el ciclo de vida, y se ejecuta únicamente si se trata de la primera actualización del componente. Normalmente se utiliza este método cuando queremos realizar una acción única tras el renderizado inicial del componente.



Updated recibe el parámetro del *map* de propiedades actualizadas y se utiliza para definir acciones que requieren que el componente esté actualizado y que se tienen que realizar, o al menos comprobar si se tienen que realizar o no cada vez que se actualiza el componente (aquí difiere del método *firstUpdated*).



UpdateComplete es una promesa que se resuelve cuando finaliza todo el ciclo de actualización del componente. Resulta muy útil cuando queremos ser notificados de que se ha terminado una actualización y no quedan más pendientes. Devuelve un booleano indicando *true*, si ya no hay más actualizaciones encoladas.



Ahora, te toca profundizar en la [documentación](#) para ver algunas características más que existen en el ciclo de vida y que también es interesante que conozcas.

Eventos

Con Lit podemos gestionar los eventos de manera más sencilla utilizando `@`. Es importante no confundirlo con los decoradores. Esta nomenclatura sí la podemos utilizar por defecto con JavaScript, ya que se añade en los literales HTML.

```
return html`
  <p><button @click="${this._increment}">Click Me!</button></p>
  <p>Click count: ${this.count}</p>
`;
```

En este ejemplo se puede ver que se ha añadido `@click`, lo que añade un *event listener* al *click* de ese elemento HTML y ejecuta el método privado `_increment` como *callback*. Este método se define como **método de clase**, y recibirá el evento como parámetro, igual que si implementamos un *addEventListener*.

No voy a entrar en más detalle, ya que ya hemos visto previamente todo lo referente a comunicación y en la documentación encontrarás esta información muy bien explicada. Sí que quiero hacer hincapié en que **revises la parte de delegación de eventos** para evitar tener demasiados *event listeners* dados de alta, ya que pueden llegar a ocasionar problemas de *performance*. También ten cuidado con dónde declaras los *event listeners* globales que no definas en los literales HTML: el *constructor* es el lugar preferido ya que únicamente se ejecuta una vez, pero quizás puedas necesitar añadirlo después de que el componente se haya renderizado, por tanto lo tendrás que añadir en el método *firstUpdated*, dándole un tiempo al navegador para que renderice el componente (*await new Promise((r) => setTimeout(r, 0))*).

¿Qué hemos aprendido?

A grandes rasgos hemos visto las diferentes formas de comunicación de las que disponemos para comunicar los componentes entre sí: a través de eventos, atributos y usando patrones o el contexto. También hemos visto Lit, una librería que nos aporta un montón de funcionalidades que nos permiten crear nuestros componentes **nativos** de una forma mucho más sencilla y optimizada, ya que nos ayuda a controlar mucho más el proceso de actualización de los componentes.

Es importante entender que Lit es una librería y no un *framework* como React o Angular, es decir, son utilidades que nos ayudan a crear *web components* de manera nativa. Los *web components* nativos siempre serán mucho más livianos, puesto que no añaden nada extra. Por contra, los *frameworks* nos facilitan la creación y añaden más utilidades.

Como ves, a lo largo de estas páginas te he presentado muchos conceptos y procesos, y además bastante complejos. Son temas complicados, pero importantísimos para tu desempeño profesional.

Te recomiendo que leas detenidamente y varias veces este fastbook y las documentaciones indicadas en el mismo. Es fundamental que pongas en práctica y pruebes todo lo aprendido en cada apartado del tema.

¡Enhorabuena! Fastbook superado